Contents

Volume 02 — Wireless, Web Scanning, App Inspection (L10–L18) **1**
L10: Wireless Reconnaissance & Monitoring 1
L11: WPA/WPA2 Security Assessment 2
L12: Web Server Scanning with Nikto 2
L13: OWASP ZAP — Installation & Passive Scanning 3
L14: ZAP Active Scanning & Proxy Interception 3
L15: Directory & Content Discovery 3
L16: SQL Injection Fundamentals & Manual Testing 4
L17: Automated SQL Injection with sqlmap 4
L18: Cross-Site Scripting (XSS) & Client-Side Attacks 5

Volume 02 — Wireless, Web Scanning, App Inspection (L10–L18)

> Cyber Security Tools · IGNOU/BAOU nou26_ge86 · Prof. Dr. Nilesh K. Modi
> **notes-3** — concept exam notes (what things are / what they do). Condensed from course `pdf/` volumes. No lab command dumps.

Wi-Fi assessment ideas, Nikto/ZAP/Gobuster roles, SQLi/XSS concepts — no lab command dumps.

L10: Wireless Reconnaissance & Monitoring

What it is / what it does

- Discover Wi-Fi networks, clients, and security configurations using passive and active wireless monitoring.
- **iwconfig / iw** — interface configuration
- **Airodump-ng** — capture 802.11 frames
- **Kismet** — wireless IDS

Defenses (one-liners)

- Use **WPA3** or WPA2-Enterprise (802.1X).
- Hide SSID is not security — rely on strong PSK/passphrase.
- Deploy WIDS (wireless IDS).
- Disable legacy protocols (WEP, WPA-TKIP).

Exam lines

- Discover Wi-Fi networks, clients, and security configurations using passive and active wireless monitoring.
- **iwconfig / iw** — interface configuration
- **Airodump-ng** — capture 802.11 frames
- **Kismet** — wireless IDS

L11: WPA/WPA2 Security Assessment

What it is / what it does

- Assess Wi-Fi authentication strength by capturing handshakes and testing passphrase entropy (authorized lab only).

Defenses (one-liners)

- Use passphrases ≥ 20 random characters.
- Enable **PMF** (802.11w) where supported.
- Rate-limit authentication attempts.
- Monitor for deauth floods (deauth attack detection).

Exam lines

- Assess Wi-Fi authentication strength by capturing handshakes and testing passphrase entropy (authorized lab only).

L12: Web Server Scanning with Nikto

What it is / what it does

- Open-source web server scanner that identifies outdated software, dangerous files, and misconfigurations.
- **Common findings** — Default files (/admin, /phpinfo.php)
- Outdated Apache/IIS versions
- HTTP methods enabled (PUT, TRACE)
- Missing security headers

Defenses (one-liners)

- Remove default pages and sample apps.
- Keep web server patched.
- Disable dangerous HTTP methods.
- Add security headers (CSP, X-Frame-Options).
- Rate-limit scanner traffic at WAF.

Exam lines

- Open-source web server scanner that identifies outdated software, dangerous files, and misconfigurations.
- **Common findings** — Default files (/admin, /phpinfo.php)
- Outdated Apache/IIS versions
- HTTP methods enabled (PUT, TRACE)
- Missing security headers

L13: OWASP ZAP — Installation & Passive Scanning

What it is / what it does

- OWASP Zed Attack Proxy (ZAP) is a free web app security scanner and intercepting proxy.

Defenses (one-liners)

- Fix all passive findings before active testing.
- Set HttpOnly and Secure on session cookies.
- Implement Content-Security-Policy.
- Run ZAP in CI/CD pipeline (baseline scan).

Exam lines

- OWASP Zed Attack Proxy (ZAP) is a free web app security scanner and intercepting proxy.

L14: ZAP Active Scanning & Proxy Interception

What it is / what it does

- Actively probe web apps for injection flaws, XSS, and misconfigurations.

Defenses (one-liners)

- Input validation and parameterized queries.
- WAF with virtual patching.
- Disable verbose error messages in production.
- Scope active scans to authorized URLs only.

Exam lines

- Actively probe web apps for injection flaws, XSS, and misconfigurations.

L15: Directory & Content Discovery

What it is / what it does

- Find hidden directories, backup files, and admin panels not linked in the application.

Defenses (one-liners)

- Remove backup files from web roots.
- Return consistent 404 pages (no size differences).
- Require authentication for admin paths.
- Monitor for enumeration patterns (many 404s from one IP).

Exam lines

- Find hidden directories, backup files, and admin panels not linked in the application.

L16: SQL Injection Fundamentals & Manual Testing

What it is / what it does

- SQL injection (SQLi) inserts malicious SQL via unsanitized input — manual testing confirms exploitability before automation.

Defenses (one-liners)

- **Parameterized queries / prepared statements** (primary fix).
- ORM with bound parameters.
- Least-privilege DB accounts (no FILE/SHUTDOWN).
- WAF as secondary layer; not a substitute for secure coding.

Exam lines

- SQL injection (SQLi) inserts malicious SQL via unsanitized input — manual testing confirms exploitability before automation.

L17: Automated SQL Injection with sqlmap

What it is / what it does

- sqlmap automates detection and exploitation of SQL injection vulnerabilities.
- **Safety** — Use –batch for non-interactive labs.
- Never run against production without written authorization.
- –tamper scripts for WAF evasion (awareness only).

Defenses (one-liners)

- Same as L16: parameterized queries.
- Block sqlmap User-Agent signatures at WAF.
- Monitor for UNION/SELECT patterns in query strings.
- Deploy database activity monitoring (DAM).

Exam lines

- sqlmap automates detection and exploitation of SQL injection vulnerabilities.
- **Safety** — Use –batch for non-interactive labs.
- Never run against production without written authorization.
- –tamper scripts for WAF evasion (awareness only).

L18: Cross-Site Scripting (XSS) & Client-Side Attacks

What it is / what it does

- XSS injects malicious scripts into pages viewed by other users — stored, reflected, and DOM-based variants.

Defenses (one-liners)

- **Output encoding** (HTML, JS, URL context-aware).
- Content-Security-Policy: default-src ‘self’.
- HttpOnly cookies prevent JavaScript access.
- Sanitize HTML with allowlist libraries (DOMPurify).

Exam lines

- XSS injects malicious scripts into pages viewed by other users — stored, reflected, and DOM-based variants.